

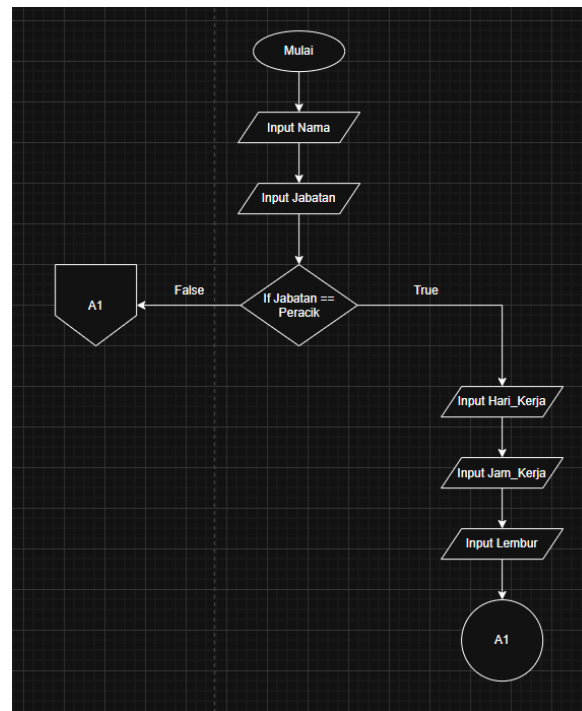
LAPORAN PRAKTIKUM
POSTTEST 3
ALGORITMA PEMROGRAMAN DASAR



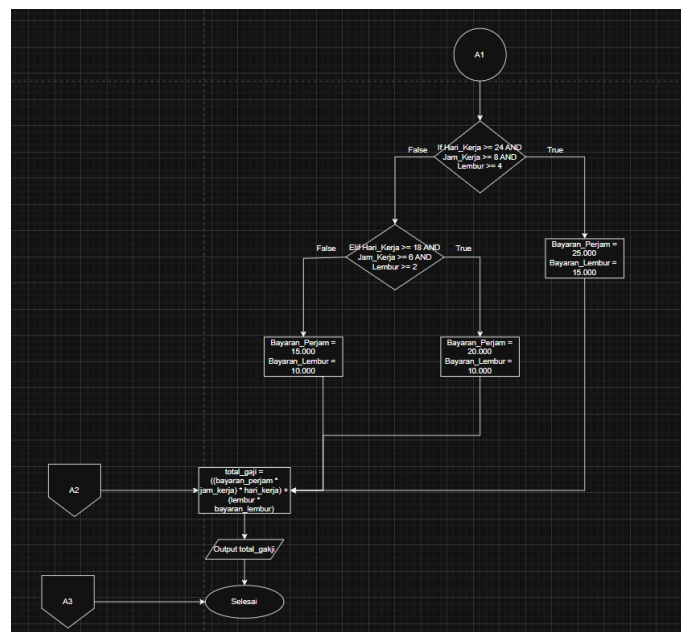
Disusun oleh:
Muhammad Fajar (2509106117)
Kelas (C2 '25)

PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN
SAMARINDA
2025

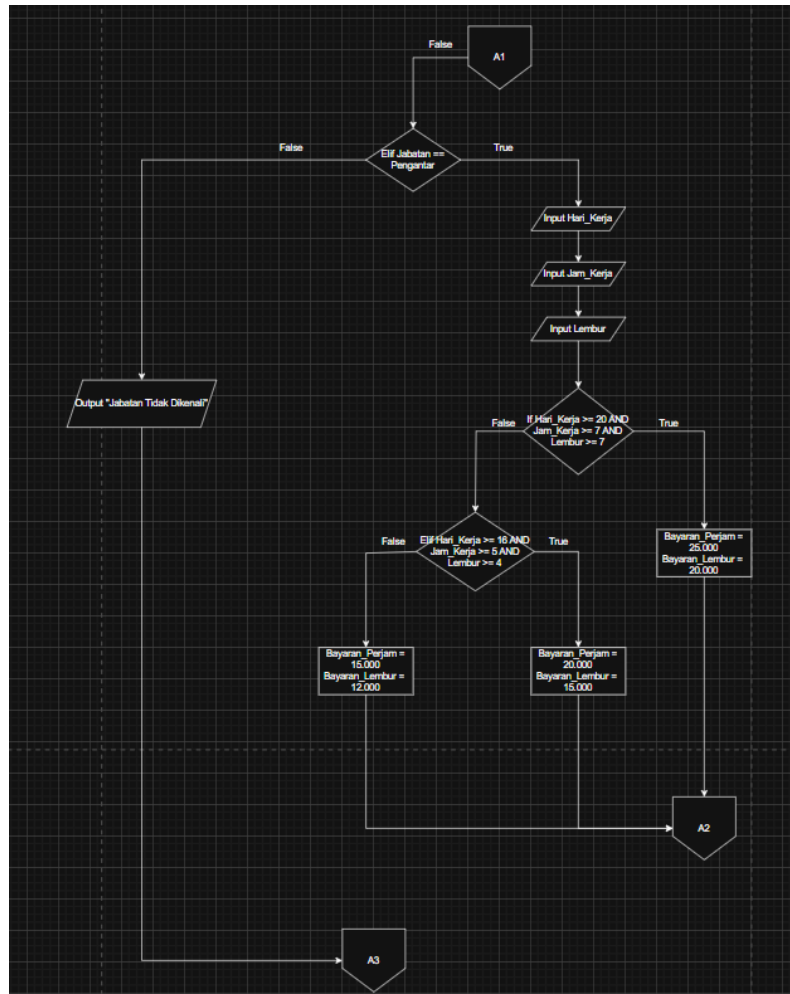
1. Flowchart



Gambar 1.1 Flowchart



Gambar 1.2 Flowchart



Gambar 1.3 Flowchart

Penjelasan:

1. Mulai
2. Input Nama
3. Input Jabatan
4. Maka program akan memeriksa, jika Jabatan == “Peracik”
5. Input Hari_Kerja
6. Input Jam_Kerja
7. Input Lembur
8. Jika true dan memeriksa program ini true, maka masuk ke eksekusi program ke If Hari_Kerja >= 24 AND Jam_Kerja >= 8 AND Lembur >= 4
9. Maka Bayaran_Perjam = 25.000, Bayaran_Lembur = 15.000
10. Hitung Total_Gaji = ((Bayaran_Perjam * Jam_Kerja) * Hari_Kerja) + (Lembur * Bayaran_Lembur))
11. Output (nama, jabatan, hari kerja, jam kerja, lembur, total gaji)
12. Jika false dan memeriksa program ini, maka masuk ke eksekusi program ke If Hari_Kerja >= 18 AND Jam_Kerja >= 6 AND Lembur >= 2
13. Maka Bayaran_Perjam = 20.000, Bayaran_Lembur = 10.000
14. Hitung Total_Gaji = ((Bayaran_Perjam * Jam_Kerja) * Hari_Kerja) + (Lembur * Bayaran_Lembur))

15. Output (nama, jabatan, hari kerja, jam kerja, lembur, total gaji)
16. Jika false
17. Maka Bayaran_Perjam = 15.000, Bayaran_Lembur = 10.000
18. Hitung Total_Gaji = ((Bayaran_Perjam * Jam_Kerja) * Hari_Kerja) + (Lembur * Bayaran_Lembur))
19. Output (nama, jabatan, hari kerja, jam kerja, lembur, total gaji)
20. Maka program akan memeriksa, jika Jabatan == "Pengantar"
21. Input Hari_Kerja
22. Input Jam_Kerja
23. Input Lembur
24. Jika true dan memeriksa program ini true, maka masuk ke eksekusi program ke If Hari_Kerja >= 20 AND Jam_Kerja >= 7 AND Lembur >= 7
25. Maka Bayaran_Perjam = 25.000, Bayaran_Lembur = 10.000
26. Hitung Total_Gaji = ((Bayaran_Perjam * Jam_Kerja) * Hari_Kerja) + (Lembur * Bayaran_Lembur))
27. Output (nama, jabatan, hari kerja, jam kerja, lembur, total gaji)
28. Jika false dan memeriksa program ini, maka masuk ke eksekusi program ke If Hari_Kerja >= 16 AND Jam_Kerja >= 5 AND Lembur >= 4
29. Maka Bayaran_Perjam = 25.000, Bayaran_Lembur = 20.000
30. Hitung Total_Gaji = ((Bayaran_Perjam * Jam_Kerja) * Hari_Kerja) + (Lembur * Bayaran_Lembur))
31. Output (nama, jabatan, hari kerja, jam kerja, lembur, total gaji)
32. Jika false
33. Maka Bayaran_Perjam = 15.000, Bayaran_Lembur = 12.000
34. Hitung Total_Gaji = ((Bayaran_Perjam * Jam_Kerja) * Hari_Kerja) + (Lembur * Bayaran_Lembur))
35. Output (nama, jabatan, hari kerja, jam kerja, lembur, total gaji)
36. Jika Jabatan tidak sesuai, maka false
37. Output ("Jabatan Tidak Bisa Dikenali")
38. Selesai

2. Deskripsi Singkat Program

A. Tujuan Program

Program ini dibuat untuk menghitung gaji karyawan PT. BOM berdasarkan jabatan, jumlah hari kerja, jam kerja per hari, serta jumlah lembur yang dilakukan.

B. Fungsi Utama

1. Memberikan perhitungan gaji yang adil sesuai jabatan dan beban kerja karyawan.
2. Mempermudah perusahaan dalam menentukan total gaji karyawan secara otomatis.
3. Mengurangi kemungkinan kesalahan perhitungan gaji karena proses dilakukan dengan bantuan program.
4. Menampilkan rincian gaji secara lengkap, mulai dari bayaran per jam, bayaran lembur, hingga total gaji yang diterima

3. Source Code

```
if jabatan == "peracik":
    hari_kerja = int(input("Masukkan jumlah hari kerja: "))
    jam_kerja = int(input("Masukkan jumlah jam kerja per hari: "))
    lembur = int(input("Masukkan jumlah lembur: "))
    if hari_kerja >= 24 and jam_kerja >= 8 and lembur >= 4:
        bayaran_perjam = 25000
        bayaran_lembur = 15000
    elif hari_kerja >= 18 and jam_kerja >= 6 and lembur >= 2:
        bayaran_perjam = 20000
        bayaran_lembur = 10000
    else:
        bayaran_perjam = 15000
        bayaran_lembur = 10000

elif jabatan == "pengantar":
    hari_kerja = int(input("Masukkan jumlah hari kerja: "))
    jam_kerja = int(input("Masukkan jumlah jam kerja per hari: "))
    lembur = int(input("Masukkan jumlah lembur: "))
    if hari_kerja >= 20 and jam_kerja >= 7 and lembur >= 7:
        bayaran_perjam = 25000
        bayaran_lembur = 20000
    elif hari_kerja >= 16 and jam_kerja >= 5 and lembur >= 4:
        bayaran_perjam = 20000
        bayaran_lembur = 15000
    else:
        bayaran_perjam = 15000
        bayaran_lembur = 12000
else:
    print("Jabatan tidak dikenali, gaji tidak bisa dihitung.")
    exit()

total_gaji = ((bayaran_perjam * jam_kerja) * hari_kerja) + (lembur * bayaran_lembur)
```

4. Uji Coba dan Hasil Output

```
PS C:\Users\sarid> & C:/Users/sarid/AppData/Local/Program
=== Penghitung Gaji Karyawan PT. BOM ===
Masukkan nama karyawan: Muhammad Fajar
Masukkan jabatan karyawan (peracik/pengantar): peracik
Masukkan jumlah hari kerja: 24
Masukkan jumlah jam kerja per hari: 8
Masukkan jumlah lembur: 4

=== Hasil Perhitungan Gaji ===
Nama Karyawan      : Muhammad Fajar
Jabatan            : Peracik
Hari Kerja         : 24 hari
Jam Kerja          : 8 jam/hari
Jumlah Lembur      : 4 kali
Bayaran Per Jam    : Rp25,000
Bayaran Lemburan   : Rp15,000
Total Gaji         : Rp4,860,000
PS C:\Users\sarid>
```

Gambar 4.1 Hasil Output

=== Penghitung Gaji Karyawan PT. BOM ===

Masukkan nama karyawan: Muhammad Fajar

Masukkan jabatan karyawan (peracik/pengantar): peracik

Masukkan jumlah hari kerja: 24

Masukkan jumlah jam kerja per hari: 8

Masukkan jumlah lembur: 4

=== Hasil Perhitungan Gaji ===

Nama Karyawan : Muhammad Fajar

Jabatan : Peracik

Hari Kerja : 24 hari

Jam Kerja : 8 jam/hari

Jumlah Lembur : 4 kali

Bayaran Per Jam : Rp25,000

Bayaran Lemburan : Rp15,000

Total Gaji : Rp4,860,000

5. Langkah-Langkah Git

5.1. Git Add

```
PS D:\Muhammad Fajar\Kuliah\APD\Pratikum\Posstest-2> git add .
```

Gambar 5.1.1 Git Add

Perintah `git add .` digunakan untuk menambahkan semua perubahan file yang ada di dalam folder proyek ke dalam staging area Git. Staging area adalah tempat sementara di mana perubahan file disiapkan sebelum benar-benar disimpan ke dalam riwayat repository melalui perintah `git commit`.

Tanda titik (.) setelah `git add` berarti semua file yang ada di direktori saat ini, termasuk subfolder di dalamnya, akan ikut ditambahkan. Jadi, jika ada file baru, file yang sudah diubah, atau file yang dihapus, semuanya akan masuk ke dalam staging area sekaligus.

Contohnya, ketika kita selesai mengedit beberapa file sekaligus, daripada menambahkan file satu per satu dengan `git add namafile`, kita bisa langsung mengetik `git add .` agar semua perubahan tercatat dalam satu langkah. Setelah itu, barulah perubahan tersebut bisa disimpan secara permanen menggunakan `git commit`.

Namun, karena `git add .` menambahkan seluruh perubahan, kita perlu berhati-hati agar tidak secara tidak sengaja menyertakan file yang sebenarnya tidak ingin dimasukkan ke dalam repository. Untuk menghindari hal ini, biasanya digunakan file `.gitignore` agar Git mengabaikan file tertentu.

5.2. Git Commit

```
PS D:\Muhammad Fajar\Kuliah\APD\Pratikum\Posstest-2> git commit -m "Memasukkan Ke Github"
[master (root-commit) fafc951] Memasukkan Ke Github
2 files changed, 68 insertions(+)
create mode 100644 kelas/pertemuan-2/pertemuan-2.py
create mode 100644 post-test/post-test-apd-2/2509106117-MuhammadFajar-PT-2.py
```

Gambar 5.2.1 Git Commit

Commit dalam Git dapat diibaratkan seperti menyimpan catatan atau rekaman atas perubahan yang telah dilakukan pada proyek. Setiap kali kita melakukan commit, Git akan menyimpan kondisi file yang sudah dimasukkan ke staging area sebagai satu titik riwayat baru. Titik riwayat ini nantinya bisa dilihat, dilacak, bahkan dikembalikan lagi jika dibutuhkan.

Sebuah commit biasanya disertai dengan pesan (commit message) yang menjelaskan apa perubahan yang dilakukan. Misalnya, ketika menambahkan fitur baru, memperbaiki bug, atau mengubah bagian tertentu dari kode. Pesan commit ini sangat penting agar orang lain — atau bahkan diri kita sendiri di kemudian hari — dapat memahami tujuan dari perubahan yang dibuat.

Setiap commit memiliki identitas unik berupa hash (serangkaian angka dan huruf) yang membuat Git bisa membedakan satu commit dengan commit lainnya. Dengan adanya commit, kita bisa menelusuri riwayat proyek dari awal hingga kondisi terbaru, serta berkolaborasi dengan lebih teratur.

5.3. Git Push

```
PS D:\Muhammad Fajar\Kuliah\APD\Pratikum\Posstest-2> git push -u origin main
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 8 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (8/8), 1.26 KiB | 184.00 KiB/s, done.
Total 8 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/Celrik08/praktikum-apd.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Gambar 5.3.1 Git Push

git push adalah perintah yang digunakan untuk mengirimkan atau mengunggah commit dari repository lokal ke repository remote, misalnya ke GitHub, GitLab, atau Bitbucket. Dengan melakukan push, semua perubahan yang sudah kita simpan melalui commit di komputer akan tersalin ke repository yang ada di server sehingga bisa diakses oleh orang lain atau digunakan pada perangkat lain.

Pada contoh di atas, origin adalah nama remote (default ketika kita menambahkan repository GitHub), dan main adalah nama branch yang akan dikirim. Artinya, semua commit yang ada di branch main pada komputer kita akan diunggah ke branch main di repository remote.

Tambahan -u (atau --set-upstream) berfungsi agar pada push berikutnya kita cukup mengetik git push tanpa harus menuliskan nama remote dan branch lagi, karena Git sudah mengingat pengaturan tersebut.

Dengan kata lain, git push adalah cara kita membagikan hasil kerja dari repository lokal ke repository online, sehingga proyek bisa ter-update dan mudah dikelola bersama tim.